

Interrupts and Event-Driven Programming

Introduction to ARM Cortex-M Microprocessors — Chapter 9

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

- 1** Why This Matters
- 2** Polling and Interrupts
- 3** The Vector Table and NVIC
- 4** Handling External Interrupts
- 5** Writing a Handler
- 6** Under the Hood
- 7** Design Challenge

- 1** Why This Matters
- 2 Polling and Interrupts
- 3 The Vector Table and NVIC
- 4 Handling External Interrupts
- 5 Writing a Handler
- 6 Under the Hood
- 7 Design Challenge

In the last chapter, Experiment 6 measured reaction time. The program sat in a loop and kept checking the button. While it waited, the processor could not do anything else: no display update, no sensor reading, no communication. It also could not sleep. A battery-powered device built this way would drain its battery fast, even while waiting for an event that might not happen for minutes. This is called **polling**, and it does not scale to real systems. This chapter shows a better way: let the hardware **interrupt** the processor only when something actually happens. This is how real embedded systems are built.

After completing this chapter, you should be able to:

1. Contrast the polling and interrupt models and justify the choice of each.
2. Explain the role of the vector table and how the NVIC dispatches an interrupt.
3. Describe the sequence of events from an external pin change to the execution of an ISR.
4. Configure the EXTI controller and SYSCFG to route a GPIO pin to an interrupt line.
5. Write an ISR that correctly clears its pending flag and shares data with the main program.
6. Use interrupt priorities and masking registers to control preemption.

- 1 Why This Matters
- 2 **Polling and Interrupts**
- 3 The Vector Table and NVIC
- 4 Handling External Interrupts
- 5 Writing a Handler
- 6 Under the Hood
- 7 Design Challenge

Polling and Interrupts

Polling

```
while (1) {  
    if (button_pressed())  
        do_something();  
}
```

- The processor **repeatedly checks** for the event
- Simple to write and understand
- Processor time is spent checking even when nothing happens
- Response time depends on how often the event is checked

Interrupts

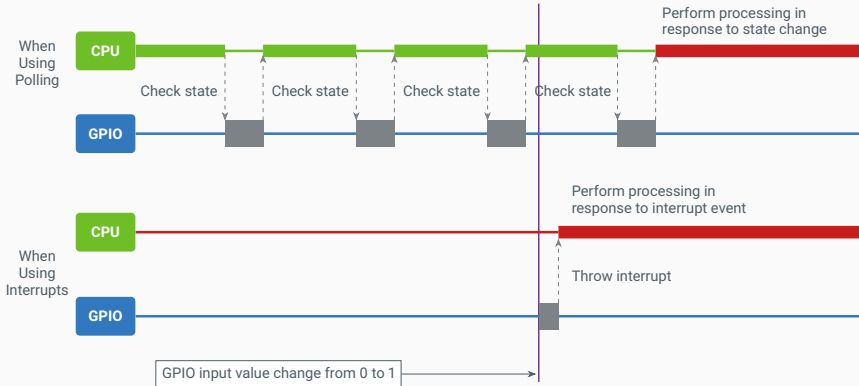
```
void EXTI15_10_IRQHandler(void) {  
    do_something();  
}
```

- Hardware signals the processor when the event occurs
- The processor can sleep or do other work meanwhile
- Response does not depend on continuously checking the event

The key difference

Polling asks "Has it happened yet?" repeatedly. An interrupt lets the hardware say "It has happened."

Interrupts vs. Polling, on the Wire



- With polling, the CPU keeps checking the state, whether or not anything has changed. It only notices a change on its next check.
- With interrupts, the CPU does nothing until the GPIO line actually changes — **one** response, exactly when needed.

When Should I Use Each?

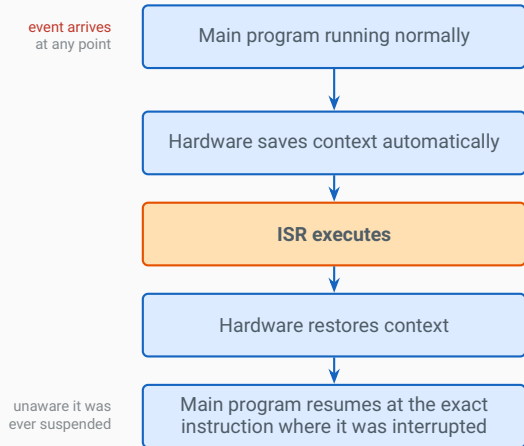
Both methods exist for good reasons. The real question is not **which is better**, but **what the code needs to ask**: “is this true right now?” (polling), or “tell me when this becomes true” (interrupt).

Decision factor	Favours interrupts	Favours polling
Timing of the event	Asynchronous, unpredictable	Periodic, or always true
Response deadline	Latency-sensitive	Non-critical
Power budget	Must sleep between events	Actively running anyway
Setup cost	Worth the NVIC/EXTI setup	A checking loop is enough

A button press happens at random times and needs a fast response, so **interrupts win**. A sensor you already read every loop pass needs nothing extra, so **polling is simplest**.

What Happens When an Interrupt Fires

We use an **external** interrupt: a button on a pin.



The processor saves this context **for you**, in hardware, before your handler runs.

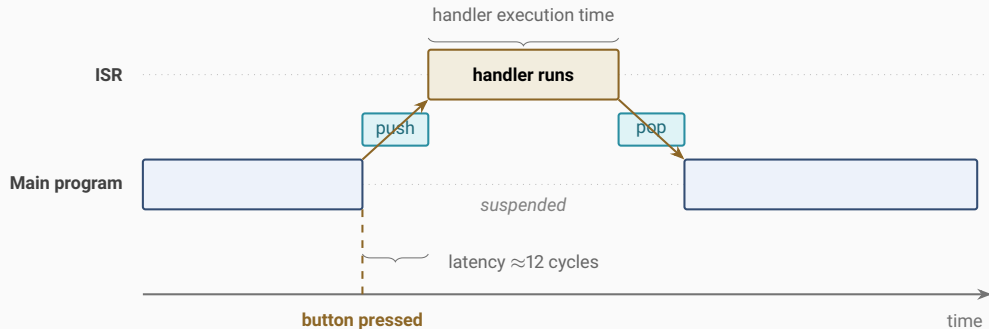
Chapter 7's stack, used automatically

It pushes R0 - R3, R12, LR, PC and xPSR — **eight words** — and pops them back on return.

Your interrupted code never notices this happened.

These eight are the **caller-saved** registers from the AAPCS. Your handler must save anything else it uses — the C compiler does this automatically.

The Same Interrupt, Viewed Against Time



Only one row runs at any instant. The main program is **paused** for the whole shaded span, then resumes at the exact instruction it was about to run. Two numbers matter here: the **latency** before the handler starts, and the handler's own **execution time** — how long the main program must wait.

- 1 Why This Matters
- 2 Polling and Interrupts
- 3 The Vector Table and NVIC**
- 4 Handling External Interrupts
- 5 Writing a Handler
- 6 Under the Hood
- 7 Design Challenge

You have seen the event happen, and what the hardware does around it. One question is still open: **how does the processor know which function to call?** The answer is a chain of five links. This section, and the next, explain the middle three.



1. **EXTI** notices the pin change.
2. The **NVIC** looks at all pending interrupts and picks a winner.
3. The **vector table** says where that winner's code lives.
4. The processor jumps there.
5. Every interrupt uses these same five links, no matter its source.

A table at the **start of flash memory**. It holds the address of every handler.

Offset	Contains
0x00	Initial SP value
0x04	Reset handler
0x08	NMI handler
0x0C	HardFault handler
...	...
0x3C	SysTick handler
0x40+	The device interrupts

Where the initial SP comes from

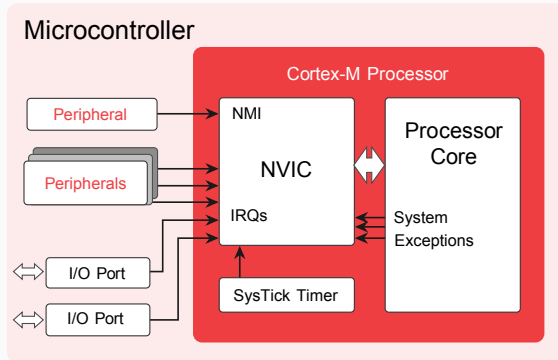
The **first word** holds the starting stack pointer. The hardware loads this automatically at reset, before a single instruction runs.

When an interrupt fires, the processor looks up its handler here and jumps to it. This is why your handler must have **exactly the right name**. The startup file maps that name into the table.

Address	Vector table entry	Points to
0x08000000	Initial SP	➤ top of stack
0x08000004	Reset	➤ Reset_Handler
0x08000008	NMI	➤ NMI_Handler
0x0800000C	HardFault	➤ HardFault_Handler
	⋮	
0x08000058	EXTIO	➤ EXTIO_IRQHandler
	⋮	
	EXTI15_10	➤ EXTI15_10_IRQHandler

matched to your code by symbol name

Two things follow from this. First, the linker matches your handler to a slot by **symbol name**. Misspell it, and a silent do-nothing handler runs instead of yours. Second, every entry's low bit is set to 1 to select Thumb state — this is why handler addresses look odd in a debugger.



Every interrupt source feeds into the NVIC: external peripherals, I/O ports, the **SysTick timer**, even an NMI. The NVIC picks exactly one at a time and hands it to the **Processor Core**, alongside the core's own **system exceptions** (HardFault and the rest).

CMSIS function	Does
<code>NVIC_EnableIRQ(n)</code>	Enable interrupt n
<code>NVIC_DisableIRQ(n)</code>	Disable it
<code>NVIC_SetPriority(n,p)</code>	Set its priority

Nested means a higher-priority interrupt can interrupt a handler that is already running.

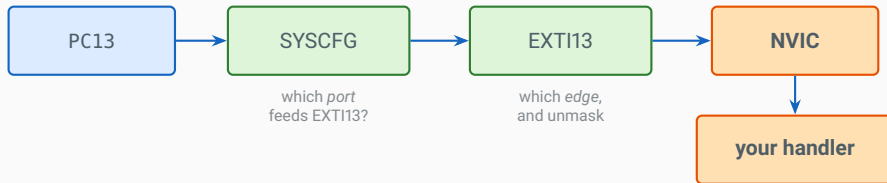
Vectored means the hardware jumps straight to the right handler. There is no software dispatch step.

Lower number means higher priority

Priority 0 is the most urgent. This feels backwards at first, and it is a common source of bugs.

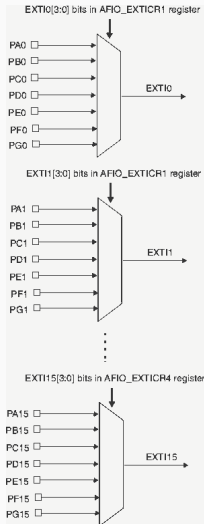
`__disable_irq()` and `__enable_irq()` mark a **critical section**: a piece of code that must not be interrupted once it starts.

- 1 Why This Matters
- 2 Polling and Interrupts
- 3 The Vector Table and NVIC
- 4 Handling External Interrupts**
- 5 Writing a Handler
- 6 Under the Hood
- 7 Design Challenge



Four things to configure, in order: Route, Trigger, Enable, Handle

1. **Route (SYSCFG)** – choose which port's pin 13 drives EXTI13
2. **Trigger (EXTI)** – select the trigger edge, then unmask the line
3. **Enable (NVIC)** – enable the interrupt and set its priority
4. **Handle** – write the handler, with the name the vector table expects



All Px0 pins feed EXTI0. All Px1 pins feed EXTI1, and so on. But **only one** pin per number can be selected at a time.

A design constraint, not a small detail

You cannot have PA0 *and* PB0 both generating interrupts at the same time.

Interrupt-capable inputs must use **different pin numbers**. This is a pin-assignment decision you make on paper, before the board is built.

SYSCFG->EXTICR[x] chooses which port drives each line: four bits per line, four lines per register.

The Configuration Sequence

```

1 // 0. Clocks
2 RCC->AHB1ENR |= (1U << 2);           // GPIOC
3 RCC->APB2ENR |= (1U << 14);          // SYSCFG
4
5 // 1. ROUTE: PC13 to EXTI13 (EXTICR[3], bits 7:4, value 0010 = port C)
6 SYSCFG->EXTICR[3] &= ~(0xFU << 4);
7 SYSCFG->EXTICR[3] |= (0x2U << 4);
8
9 // 2. TRIGGER: falling edge -- an active-low button goes HIGH->LOW
10 EXTI->FTSR |= (1U << 13);
11 EXTI->IMR  |= (1U << 13);           // unmask the line
12
13 // 3. ENABLE in the NVIC
14 NVIC_SetPriority(EXTI15_10_IRQn, 2);
15 NVIC_EnableIRQ(EXTI15_10_IRQn);
16
17 // 4. HANDLE: write EXTI15_10_IRQHandler -- next section

```

Lines 10–15 all share one handler, EXTI15_10_IRQHandler. This is why the handler must check *which* line fired.

- 1 Why This Matters
- 2 Polling and Interrupts
- 3 The Vector Table and NVIC
- 4 Handling External Interrupts
- 5 Writing a Handler**
- 6 Under the Hood
- 7 Design Challenge

```
volatile uint8_t button_event = 0;    // shared with main() -> volatile

void EXTI15_10_IRQHandler(void)
{
    if (EXTI->PR & (1U << 13)) {      // was it line 13?
        EXTI->PR = (1U << 13);        // clear it by writing 1
        button_event = 1;             // just record it; main() acts
    }
}
```

Three rules, and each one matters

1. The **name** must match the vector table exactly.
2. **Clear the pending flag**, by writing 1 to it. Skip this, and the handler runs forever.
3. Anything shared with `main()` must be **volatile**.

Keep handlers short

- Clear the flag
- Set a variable, or move one value
- Return

Avoid: long loops, `printf`, or anything that waits for another interrupt.

Think about latency, not lines of code.

Two hazards

- **Missing volatile:** the optimiser caches a value that the ISR is changing
- **Torn reads:** `main()` reads a multi-byte variable while the ISR is halfway through writing it

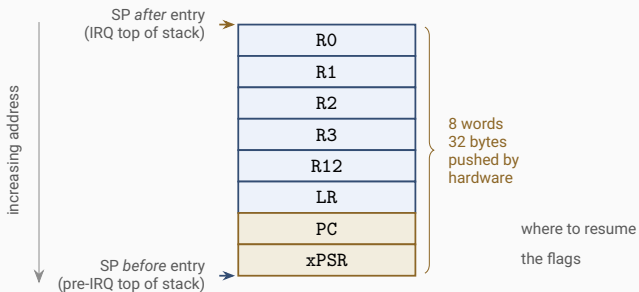
The pattern you just saw

This is exactly what `button_event` did: the handler sets a **flag**, and `main()` does the real work (toggling the LED) the next time it checks. For anything wider than a byte, briefly disable interrupts while reading it – or use a single `uint32_t`, which the processor always reads atomically.

- 1 Why This Matters
- 2 Polling and Interrupts
- 3 The Vector Table and NVIC
- 4 Handling External Interrupts
- 5 Writing a Handler
- 6 Under the Hood**
- 7 Design Challenge

The Exception Stack Frame

The “save context, run handler, restore context” step is just an ordinary **push and pop**. Hardware does it, not an instruction you wrote. Eight words go onto the stack on entry, and the same eight come off on return.



- PC and xPSR make resuming possible: one says **where** to continue, the other restores the flags
- Your interrupted code never notices, and needs no special setup code of its own

- Each interrupt request line is an **edge-triggered flip-flop**: a peripheral event sets it, and clearing the pending bit resets it.
- The NVIC examines these pending requests using **priority-encoding logic**—the same priority encoder used to resolve simultaneous requests.
- The winning index selects the corresponding entry in the vector table.

Why a forgotten pending-flag clear hangs the system

If your code never resets that flip-flop, the request remains asserted. When the handler returns, the NVIC sees that the line is still pending and immediately invokes the handler again. The result is **an interrupt handler that runs forever**—one of the most common interrupt bugs. This is why the handler you already wrote contains `EXTI->PR = (1U << 13);`.

You will rarely write these directly, since CMSIS wraps them. But it is worth knowing them, since they live at 0xE000E100:

Register	Does
NVIC_ISER[x]	Set-enable: writing 1 enables that interrupt; writing 0 does nothing
NVIC_ICER[x]	Clear-enable: writing 1 disables
NVIC_IPR[x]	Priority for each channel

Separate set and clear registers avoid a read-modify-write race. **This is the same design idea as GPIO's BSRR.**

Once an event is interrupt-driven, the main loop often has nothing to do until the next one arrives. **WFI** (“wait for interrupt”) lets the CPU stop instead of spinning:

```
while (1) {  
    __WFI();    // sleep until an interrupt wakes the CPU  
}
```

- The core’s clock halts; the NVIC keeps watching for a pending interrupt
- On the next interrupt, the core wakes and the usual exception entry sequence runs, exactly as if it had been running
- **Interrupts** remove the need to **poll** for an event; **WFI** removes the need to **keep running** while waiting for one

A battery-powered sensor node can sleep almost all the time and wake only when a reading is due.

- **Polling** wastes the processor and ties response time to the loop. **Interrupts** give a fast, bounded response and let the CPU sleep.
- On an interrupt, the processor **automatically** pushes eight registers, then restores them on return.
- The **vector table** holds every handler's address, and the initial SP in its first word.
- A pin reaches the core through **SYSCFG** → **EXTI** → **NVIC**. All Pxn pins share EXTI_n, so only one may be selected.
- In the handler: match the **name**, **clear the pending flag**, and mark shared variables **volatile**.
- **Keep handlers short**. Set a flag; let `main()` do the real work.

Next: Chapter 10 – Timers, Timing and PWM

`delay_ms()` is wrong in three ways. Next, we learn to do timing properly.

- 1 Why This Matters
- 2 Polling and Interrupts
- 3 The Vector Table and NVIC
- 4 Handling External Interrupts
- 5 Writing a Handler
- 6 Under the Hood
- 7 Design Challenge**

The brief

A system has four event sources: a 1 kHz sensor-ready signal, an emergency-stop button, a 50 Hz zero-crossing detector, and a rotary encoder producing up to 2000 edges per second.

For each source, decide whether it should be polled or interrupt-driven. Assign interrupt priorities to the ones you choose to make interrupt-driven. Justify each decision using the latency requirement and the cost of servicing it. Then find the pair of sources most likely to interfere with each other, and say what evidence would show you the problem had happened.

Questions?

Introduction to ARM Cortex-M Microprocessors — Chapter 9